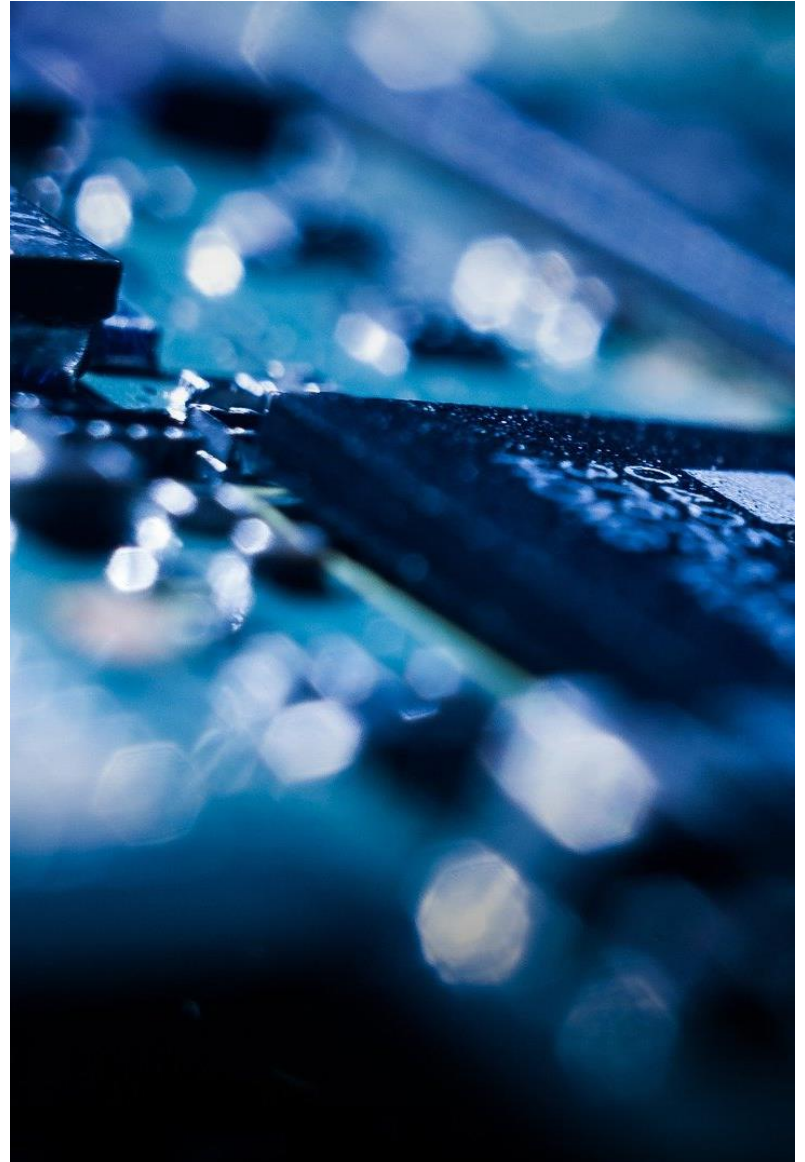


Programmierung eingebetteter Systeme

Allgemeine Sprachelemente in
der C Programmierung

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



Überblick – Allgemeine Sprachelemente

- ▶ Einführung
- ▶ Programmbeispiel
- ▶ Datentypen
- ▶ Operatoren und Kontrollstrukturen

Hardwarenahe C-Programmierung

Assembler: Prozessor optimal nutzen (Register, Spezialregister, Speicherverwaltung usw.)

Probleme: Jede Prozessorfamilie hat eine etwas andere Assemblersprache abhängig von der Hardware des Prozessors (Register, Spezialregister, Speicherverwaltung usw.)

Bei größeren Programmen geht die Übersicht leicht verloren.

...

Einheitliche Programmiersprache auch für hardwarenahe Programmierung:

ANSI-C (ISO/IEC 9899 programming languages – C)

Hardwarenahe C-Programmierung

Aber: Hardwarenahe C Programme sind nicht wirklich portabel.

Keine Abstrahierung vom realen Prozessor durch ein Betriebssystem.

Benutzung von Bibliotheken eingeschränkt nur ANSI/ISO Library.

Übersetzen von C-Programmen

Quellcode-Dateien in C

`file.c`: Quell-Code
Programm
Problemlösung

`file.h`: Header-File
Variablen, Konstanten, Prototypen
Definitionen

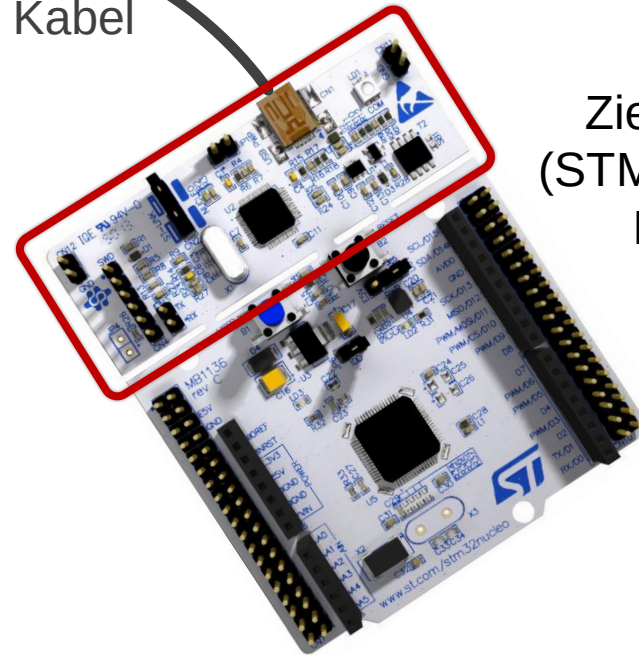
Entwicklungsumgebung

Entwicklungsrechner (PC)



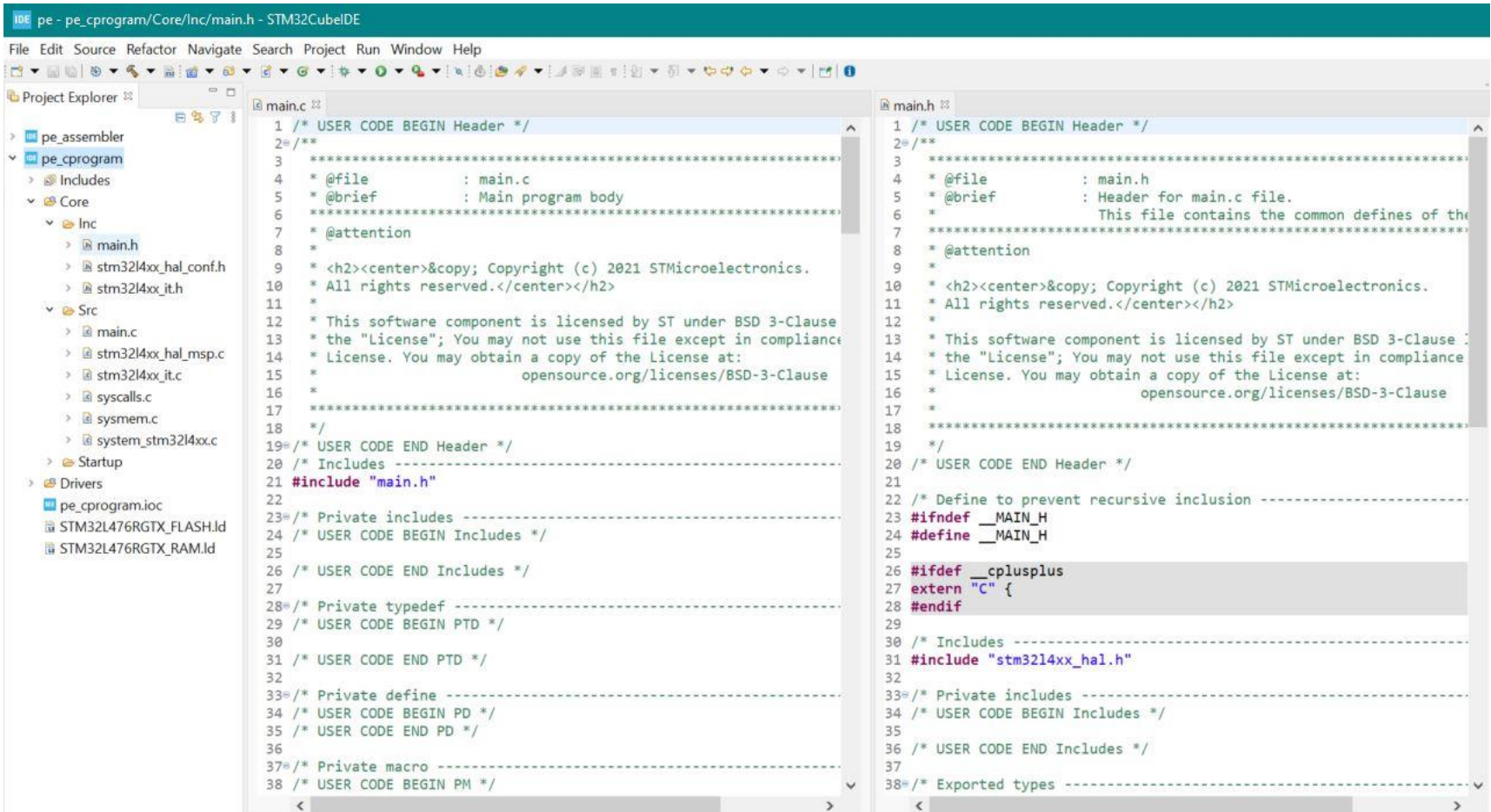
USB Kabel

**Embedded ST-LINK/V2
Programming and
Debugging Tool**



Zielrechner
(STM32 Nucleo
Board)

Entwicklungsumgebung



The screenshot displays the STM32CubeIDE development environment. The top bar shows the menu (File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, Help) and a toolbar. The Project Explorer on the left shows the project structure: pe_assembler, pe_cprogram, Includes, Core (Inc, Src), Startup, Drivers, and pe_cprogram.ioc. The main window shows two files: main.c and main.h.

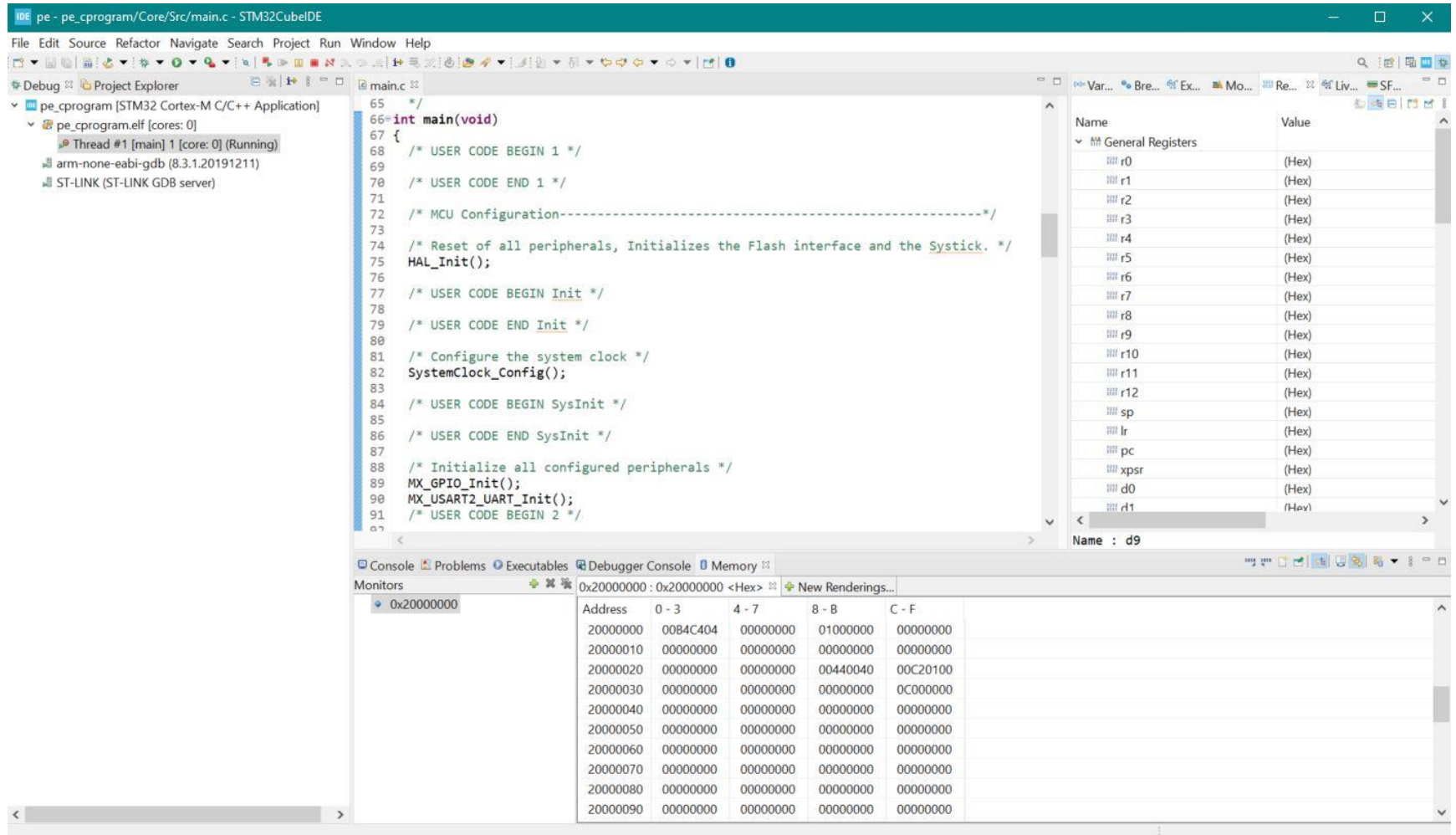
main.c

```
1 /* USER CODE BEGIN Header */
2 /**
3  * @file      : main.c
4  * @brief     : Main program body
5  * @attention
6  *
7  * <h2><center>&copy; Copyright (c) 2021 STMicroelectronics.
8  * All rights reserved.</center></h2>
9  *
10 * This software component is licensed by ST under BSD 3-Clause
11 * the "License"; You may not use this file except in compliance
12 * License. You may obtain a copy of the license at:
13 *      opensource.org/licenses/BSD-3-Clause
14 *
15 *
16 *
17 *
18 */
19 /* USER CODE END Header */
20 /* Includes -----
21 #include "main.h"
22
23 /* Private includes -----
24 /* USER CODE BEGIN Includes */
25
26 /* USER CODE END Includes */
27
28 /* Private typedef -----
29 /* USER CODE BEGIN PTD */
30
31 /* USER CODE END PTD */
32
33 /* Private define -----
34 /* USER CODE BEGIN PD */
35 /* USER CODE END PD */
36
37 /* Private macro -----
38 /* USER CODE BEGIN PM */
```

main.h

```
1 /* USER CODE BEGIN Header */
2 /**
3  * @file      : main.h
4  * @brief     : Header for main.c file.
5  *             This file contains the common defines of the
6  *
7  * @attention
8  *
9  * <h2><center>&copy; Copyright (c) 2021 STMicroelectronics.
10 * All rights reserved.</center></h2>
11 *
12 * This software component is licensed by ST under BSD 3-Clause
13 * the "License"; You may not use this file except in compliance
14 * License. You may obtain a copy of the License at:
15 *      opensource.org/licenses/BSD-3-Clause
16 *
17 *
18 *
19 */
20 /* USER CODE END Header */
21
22 /* Define to prevent recursive inclusion -----
23 #ifndef __MAIN_H
24 #define __MAIN_H
25
26 #ifdef __cplusplus
27 extern "C" {
28 #endif
29
30 /* Includes -----
31 #include "stm32l4xx_hal.h"
32
33 /* Private includes -----
34 /* USER CODE BEGIN Includes */
35
36 /* USER CODE END Includes */
37
38 /* Exported types -----
```


Entwicklungsumgebung



Programmbeispiel

Programmbeispiel

```
...
/* Private includes -----*/
#include <stdio.h>

...
/* Private define -----*/
#define VALUE 5

...
/* Private variables -----*/
int count;

...
/* Private function prototypes -----*/
void test(void);

...
int main(void)
{
    count = 0;
    ...
    /* Infinite loop */
    while (1)
    {
        count = count + VALUE;
        printf("Count: %i", count);
    }
}
...
```

Bibliotheksfunktionen

Definition von Konstanten

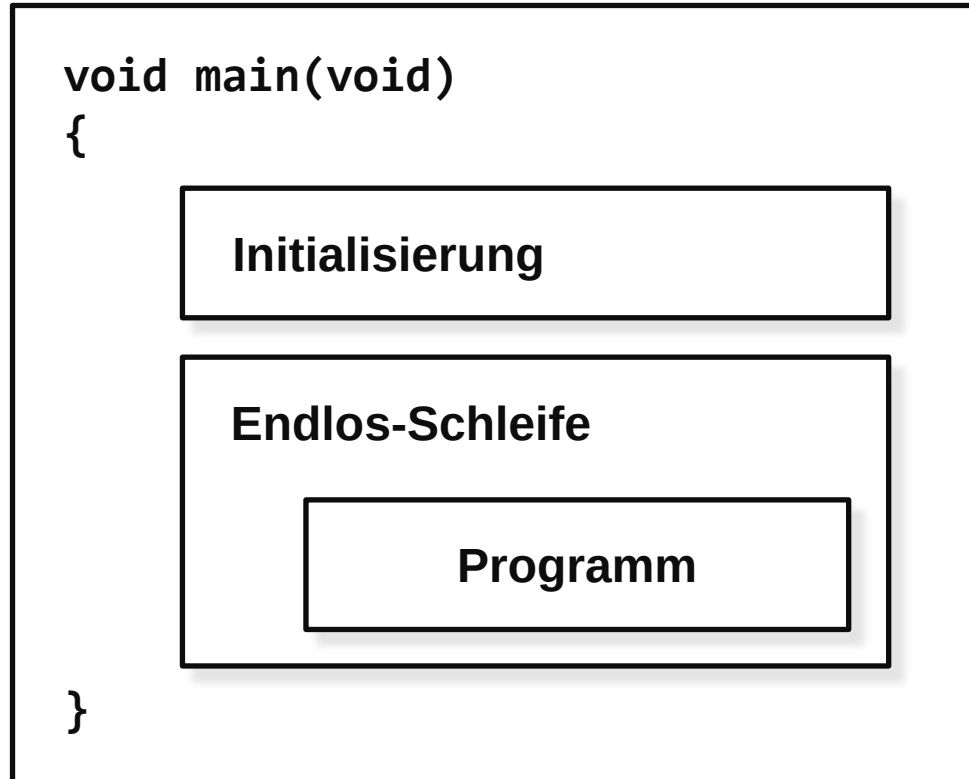
**Erzeugen von Variablen
(Speicherplatz bereitstellen)**

**Bekanntgabe von Funktionen und
Variablentypen**

„Hauptprogramm“ (erste Funktion)

Endlos-Schleife (kein Betriebssystem)

Programmstruktur *Embedded Systems*



Hardwarenahe C-Programmierung

Variablen sind "Behälter" für Daten (Zahlen, Buchstaben usw). Diesen Variablen können im Verlauf des Programms unterschiedliche Werte zugewiesen werden.

Ziel = Quelle;

```
counter = 0;
```

Datenfluss



Vergleich mit Assembler:

```
ldr r1, =0x20000000
```

Datenfluss



Ziel = Ausdruck;

```
counter = xy + 25;
```

Datenfluss



```
str r0, [r1]
```

Datenfluss



Hardwarenahe C-Programmierung

Variablen sind "Behälter" für Daten (Zahlen, Buchstaben usw). Diesen Variablen können im Verlauf des Programms unterschiedliche Werte zugewiesen werden.

Ziel = Quelle;

```
counter = 0;
```

Ziel = Ausdruck;

```
counter = xy + 25;
```

Vergleich mit Assembler:

**Verwendung des Namens einer Variablen:
Zugriff auf die Daten**

```
...
```

```
adr    r0, var
```

```
...
```

```
var: .word 0x00000005
```

Hardwarenahe C-Programmierung

Variablen sind "Behälter" für Daten (Zahlen, Buchstaben usw). Diesen Variablen können im Verlauf des Programms unterschiedliche Werte zugewiesen werden.

Ziel = Quelle;

```
counter = 0;
```

Ziel = Ausdruck;

```
counter = xy + 25;
```

Vergleich mit Assembler:

**Verwendung des Namens einer Variablen:
Zugriff auf die Daten**

...

adr r0, var

...

var: .word 0x00000005



Hardwarenahe C-Programmierung

Variablen sind "Behälter" für Daten (Zahlen, Buchstaben usw). Diesen Variablen können im Verlauf des Programms unterschiedliche Werte zugewiesen werden.

Ziel = Quelle;

```
counter = 0;
```

Vergleich mit Assembler:

```
ldr r1, =0x20000000
```

```
str r0, [r1]
```

Ziel = Ausdruck;

```
counter = xy + 25;
```

**Datentypen, Operatoren,
Zahlendarstellung**

Datentypen

Datentypen

char, signed char

unsigned char

int

unsigned int

short int

unsigned short int

long int

unsigned long int

float

double

Datentypen

char, signed char
unsigned char
int
unsigned int
short int
unsigned short int
long int
unsigned long int
float
double

Definition und Wertzuweisung von Character-Variablen

```
char pe_char;  
char buchstabe;  
...  
pe_char = '?';  
buchstabe = 'A';  
buchstabe = 0x41;
```

```
char pe_char = pe_char = '?';  
char buchstabe = 'A';
```

Datentypen

char, signed char
unsigned char

Character-Variablen

int
unsigned int
short int
unsigned short int
long int
unsigned long int
float
double

Definition und Wertzuweisung von Character-Variablen

```
char pe_char;  
char buchstabe;  
...  
pe_char = '?';  
buchstabe = 'A';  
buchstabe = 0x41;
```

```
char pe_char = pe_char = '?';  
char buchstabe = 'A';
```

Datentypen

char, signed char
unsigned char

Character-Variablen

int
unsigned int
short int
unsigned short int
long int
unsigned long int
float
double

Definition und Wertzuweisung von Character-Variablen

```
char pe_char;  
char buchstabe;  
...  
pe_char = '?';  
buchstabe = 'A';  
buchstabe = 0x41;
```

```
char pe_char = pe_char = '?';  
char buchstabe = 'A';
```

Datentypen

char, signed char
unsigned char

Character-Variablen

int
unsigned int
short int
unsigned short int
long int
unsigned long int
float
double

Definition und Wertzuweisung von Character-Variablen

```
char pe_char;  
char buchstabe;
```

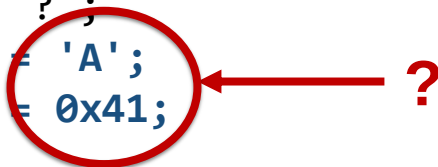
...

```
pe_char = '?';
```

```
buchstabe = 'A';
```

```
buchstabe = 0x41;
```

```
char pe_char = pe_char = '?';  
char buchstabe = 'A';
```



Datentypen

Darstellbare Zeichen

Zeichen	Hex
@	0x40
A	0x41
B	0x42
C	0x43
D	0x44
E	0x45
F	0x46
G	0x47
H	0x48
I	0x49
J	0x4A
K	0x4B
L	0x4C
M	0x4D
N	0x4E
O	0x4F

Zeichen	Hex
P	0x50
Q	0x51
R	0x52
S	0x53
T	0x54
U	0x55
V	0x56
W	0x57
X	0x58
Y	0x59
Z	0x5A
[	0x5B
\	0x5C
]	0x5D
^	0x5E
_	0x5F

Zeichen	Hex
`	0x60
a	0x61
b	0x62
c	0x63
d	0x64
e	0x65
f	0x66
g	0x67
h	0x68
i	0x69
j	0x6A
k	0x6B
l	0x6C
m	0x6D
n	0x6E
o	0x6F

Zeichen	Hex
p	0x70
q	0x71
r	0x72
s	0x73
t	0x74
u	0x75
v	0x76
w	0x77
x	0x78
y	0x79
z	0x7A
{	0x7B
	0x7C
}	0x7D
~	0x7E
DEL	0x7F

Datentypen

char, signed char
unsigned char

int

unsigned int

short int

unsigned short int

long int

unsigned long int

float

double

Größe im Speicher:

1 Byte

Vergleich mit Assembler:

char : BYTE

Definition und Wertzuweisung von Character-Variablen

```
char led_reihe_oben;
```

```
char led_reihe_unten;
```

```
...
```

```
led_reihe_oben = 0x55;
```

```
led_reihe_unten = 0x55;
```

Ausgabe der Variablen auf die Hardware
-> Pointer

(jede zweite LED leuchtet)

Datentypen

char, signed char
unsigned char

int
unsigned int
short int
unsigned short int
long int
unsigned long int

Integer-Variablen

float
double

Definition und Wertzuweisung von Integer-Variablen

```
long int pe;  
int counter;  
...  
pe = 300000;  
counter = 0;
```

```
long int pe = 3000000;  
int counter = 0;
```

Datentypen

char, signed char
unsigned char

int
unsigned int
short int
unsigned short int
long int
unsigned long int

float
double

Integer-Variablen

Größe im Speicher:

int: 2 oder 4 Byte
short int: 2 Byte
long int: 4 Byte

Vergleich mit Assembler:

short int : WORD
long int : LONG

Definition und Wertzuweisung von Integer-Variablen

Datentypen

char, signed char
unsigned char

int
unsigned int
short int
unsigned short int
long int
unsigned long int

float
double

Integer-Variablen

Größe im Speicher:

int: 2 oder 4 Byte
short int: 2 Byte
long int: 4 Byte

Vergleich mit Assembler:

short int : WORD
long int : LONG

Definition und Wertzuweisung von Integer-Variablen

```
long int var1;  
short int var2;  
...  
var1 = 0x7FFFFFFF;  
var2 = 0x7FFF;
```

Datentypen

char, signed char
unsigned char

int
unsigned int
short int
unsigned short int
long int
unsigned long int

float
double

Integer-Variablen

Größe im Speicher:

int: 2 oder 4 Byte
short int: 2 Byte
long int: 4 Byte

Vergleich mit Assembler:

short int : WORD
long int : LONG

Definition und Wertzuweisung von Integer-Variablen

```
long int var1;  
short int var2;  
...  
var1 = 0x7FFFFFFF;  
var2 = 0x7FFF;
```

```
char schalter;  
int var3;  
...  
var3 = (int)schalter;  
var1 = (long int)var2;
```

Cast-Operator
Datentypwandlung

Datentypen

char, signed char
unsigned char

int
unsigned int
short int
unsigned short int
long int
unsigned long int
float
double

Definition und Wertzuweisung von Integer-Variablen

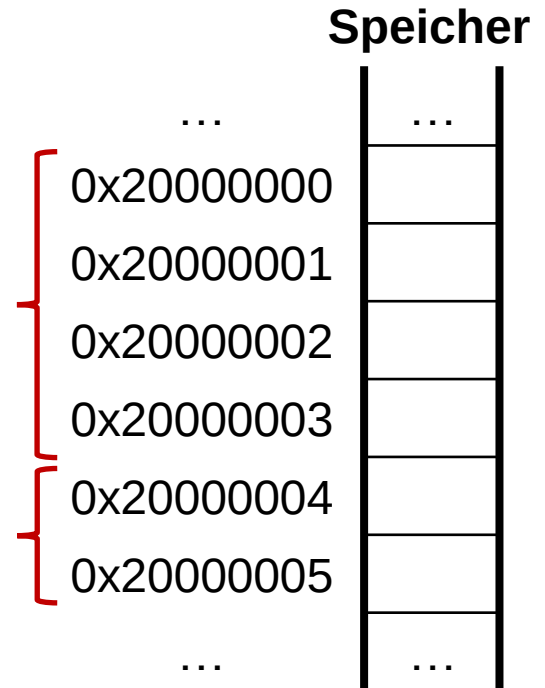
```
long int var1;  
short int var2;
```

...

```
var1 = 0x12345678;
```

```
var2 = 0x1A2B;
```

**Compiler reserviert
Speicherplatz. Beispiel:**
var1 auf 0x20000000
var2 auf 0x20000004



Datentypen

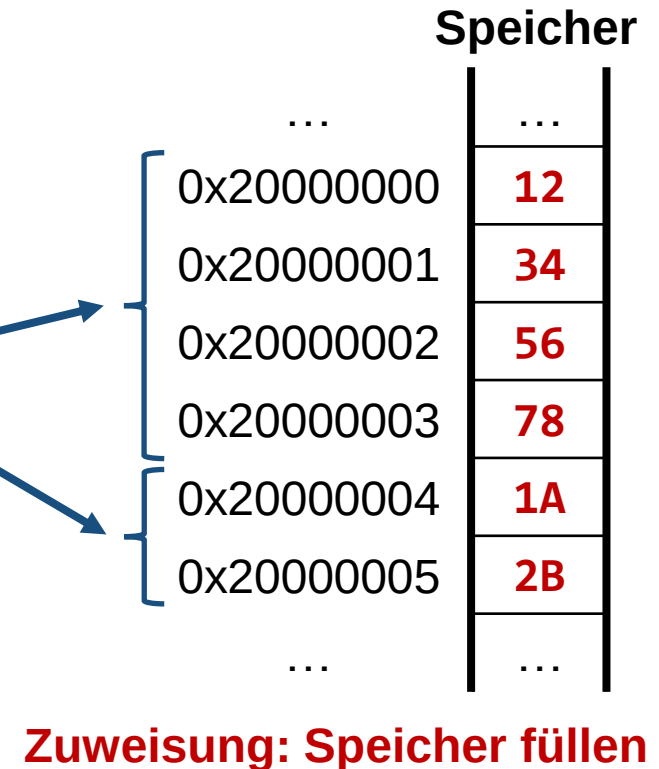
char, signed char
unsigned char
int
unsigned int
short int
unsigned short int
long int
unsigned long int
float
double

**Compiler reserviert
Speicherplatz. Beispiel:**
var1 auf 0x20000000
var2 auf 0x20000004

Definition und Wertzuweisung von Integer-Variablen

```
long int var1;  
short int var2;
```

```
var1 = 0x12345678;  
var2 = 0x1A2B;
```



Datentypen

char, signed char
unsigned char

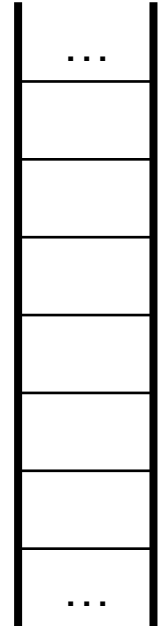
int
unsigned int
short int
unsigned short int
long int
unsigned long int

float
double

**Compiler reserviert
Speicherplatz. Beispiel:
schalter auf 0x20000000
count auf 0x20000002**

...
0x20000000
0x20000001
0x20000002
0x20000003
0x20000004
0x20000005
...

Speicher



Definition und Wertzuweisung von Integer-Variablen

```
long int var1;  
short int var2;  
...  
var1 = 0x12345678;  
var2 = 0x1A2B;
```

```
char schalter;  
short int count;  
...  
schalter = 0x7F;  
count = 15;
```

Datentypen

char, signed char
unsigned char

int
unsigned int
short int
unsigned short int
long int
unsigned long int

float
double

**Compiler reserviert
Speicherplatz. Beispiel:
schalter auf 0x20000000
count auf 0x20000002**

...
{ 0x20000000
0x20000001
0x20000002
0x20000003
0x20000004
0x20000005
...

Speicher

...
7F
00
15
...

Definition und Wertzuweisung von Integer-Variablen

```
long int var1;  
short int var2;  
...  
var1 = 0x12345678;  
var2 = 0x1A2B;
```

```
char schalter;  
short int count;
```

```
schalter = 0x7F;  
count = 15;
```

**Zuweisung:
Speicher füllen**

Datentypen

char, signed char
unsigned char
int
unsigned int
short int
unsigned short int
long int
unsigned long int

float
double

Float-Variablen

Größe im Speicher:

float: 4 Byte

double: 8 Byte

Definition und Wertzuweisung von Float-Variablen

```
float f11;
```

```
float f11 = 3.1415927;
```

```
...
```

```
f11 = 3.1415927;
```

Datentypen

Typkonvertierung: cast-Operator

Unterschiedlicher Aufwand

...

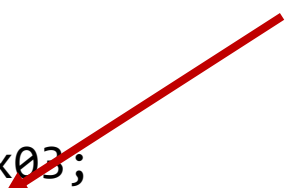
```
char      schalter_8bit;  
short int index;  
float     summe;
```

...

```
schalter_8bit = 0x03;  
index = (short int)schalter_8bit;  
summe = (float)index
```

...

Vorzeichenrichtige Erweiterung
(0x03 → 0x0003)



Datentypen

Typkonvertierung: cast-Operator

Unterschiedlicher Aufwand

...

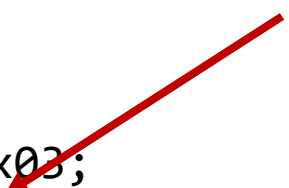
```
char      schalter_8bit;  
short int index;  
float     summe;
```

...

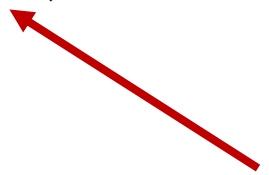
```
schalter_8bit = 0x03;  
index = (short int)schalter_8bit;  
summe = (float)index
```

...

Vorzeichenrichtige Erweiterung
(0x03 → 0x0003)



Umrechnung in anderes Intern-Format



Operatoren und Kontrollstrukturen

Arithmetische Operatoren

counter = xy + 25;

Addition

wert = a - b;

Subtraktion

durchmesser = radius * 2;

Multiplikation

div = x / y;

Division

a = -b;

Negation

count++;

Postincrement

count--;

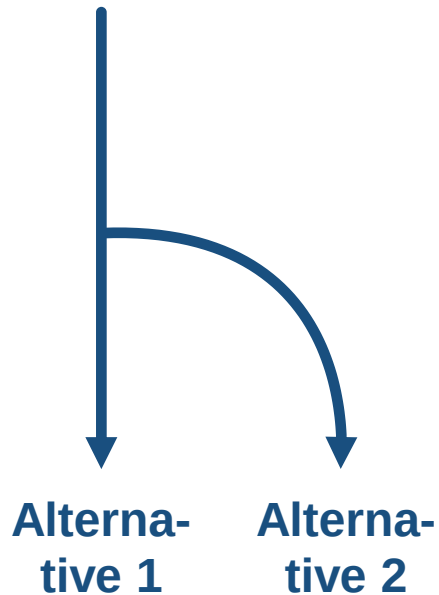
Postdecrement

Kontrollstrukturen

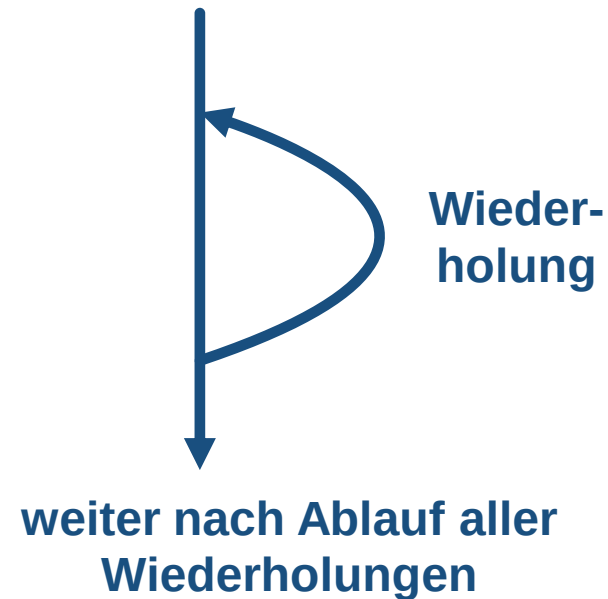
Zuweisung: nur sequentieller Programmablauf möglich

Kontrollstrukturen zur Änderung des sequentiellen Programmablaufs:

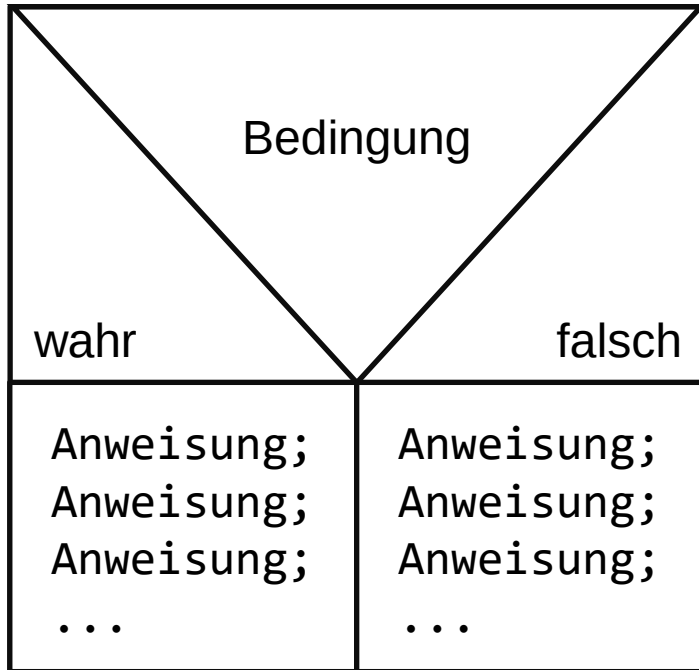
Verzweigung



Schleife



Verzweigung im Programmablauf



```
if (Bedingung)
{
    Anweisung;
    Anweisung;
    ...
}

else
{
    Anweisung;
    Anweisung;
    ...
}
```

Vergleichsoperatoren

>	größer als (greater than)
>=	größer oder gleich (greater or equal)
<	kleiner als (less than)
<=	kleiner oder gleich (less or equal)
==	gleich (equal)
!=	ungleich (not equal)

Vergleichsoperatoren

>	größer als (greater than)
>=	größer oder gleich (greater or equal)
<	kleiner als (less than)
<=	kleiner oder gleich (less or equal)
==	gleich (equal)
!=	ungleich (not equal)

Beispiele:

```
// -----  
if (count != max)  
{  
    analog_output = 2.5;  
    count++;  
}  
else  
{  
    analog_output = 5.0;  
    count = MAX_COUNT;  
}  
// -----  
if ((messwert >= minimum)  
    && (messwert <= maximum))  
{  
    ...  
}
```

Logische Operatoren

Zusammengesetzte Bedingungen

&& Logisches UND
(wenn Bedingung1 und
gleichzeitig Bedingung2
erfüllt ist)

|| logisches ODER
(wenn Bedingung1 oder
Bedingung2 erfüllt ist)

! logisches NICHT
(wenn die Bedingung
nicht erfüllt ist)

Logische Operatoren

Zusammengesetzte Bedingungen

&& Logisches UND
(wenn Bedingung1 und
gleichzeitig Bedingung2
erfüllt ist)

|| logisches ODER
(wenn Bedingung1 oder
Bedingung2 erfüllt ist)

! logisches NICHT
(wenn die Bedingung
nicht erfüllt ist)

Beispiele:

```
//-----  
if(!((count == maximum) &&  
    (lichtschranke != ON)))  
{  
    analog_output = 2.5;  
    count++;  
}  
else  
{  
    analog_output = 5.0;  
    count = MAX_COUNT;  
}  
//-----  
if(((messwert >= minimum) &&  
    (messwert <= maximum)) ||  
    (count < 25))  
{  
    ...  
}
```

Mehrfach-Alternative

Ausdruck			
case1	case2		default
Anweisung; Anweisung; Anweisung; ...	Anweisung; Anweisung; Anweisung; ...	...	Anweisung; Anweisung; Anweisung; ...

```
switch (Ausdruck)
{
    case KONSTANTE1:
        Anweisung;
        Anweisung;
        ...
        break;

    case KONSTANTE2:
        Anweisung;
        Anweisung;
        ...
        break;

    ...

    default:
        Anweisung;
        Anweisung;
        ...
        break;
}
```

Mehrfach-Alternative

```
#define TASTER_NICHT_GEDRUECKT 1
```

```
// Zustandsautomat für Tastererkennung
```

```
switch(state)
```

```
{
```

```
    // Taster ist nicht gedrückt
```

```
    case TASTER_NICHT_GEDRUECKT:
```

```
        Anweisung;
```

```
        Anweisung;
```

```
        ...
```

```
    break;
```

```
    // Wartezeit für Entprellung
```

```
    case WARTEN_ENTPRELL:
```

```
        Anweisung;
```

```
        Anweisung;
```

```
        ...
```

```
    break;
```

```
...
```

```
...
```

```
// weitere Tasterzustände
```

```
...
```

```
// falsche Zustandsnummer
```

```
default
```

```
    Anweisung;
```

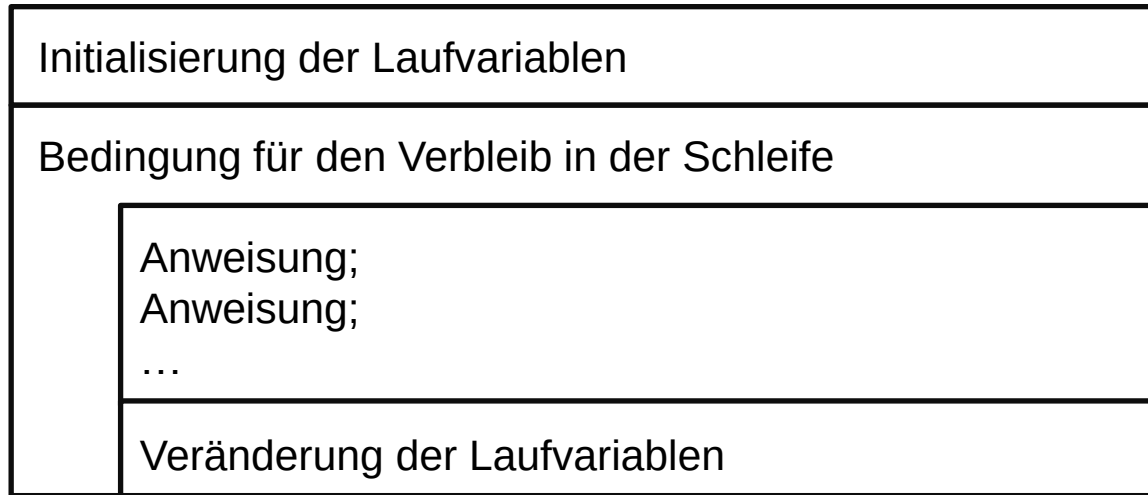
```
    Anweisung;
```

```
    ...
```

```
break;
```

```
}
```

Schleife im Programmablauf



```
for (Initialisierung der Laufvariablen;  
    Bedingung zum Verbleib in der Schleife;  
    Veränderung der Laufvariablen)  
{  
    Anweisung;  
    Anweisung;  
    ...  
}
```

Schleife im Programmablauf

Beispiele:

```
int count;
```

```
...
```

```
for (count = 0; count < 10; count++)  
{  
    Anweisung  
    Anweisung  
    ...  
}
```